AI PROMPT LIBRARY Capstone Project

CSC 489 Spring 2026

HOW TO USE THIS GUIDE

This library contains 50+ AI prompts to help you build your capstone project quickly and efficiently. Each prompt is designed to generate working code for specific features or vulnerabilities.

HOW TO USE: 1. Copy a prompt below 2. Paste into ChatGPT, Claude, or your preferred AI tool 3. Review the generated code 4. Copy into your project files 5. Test to make sure it works 6. Customize as needed

TIPS FOR BEST RESULTS: • Be specific about what you want • Mention you're using Next.js + Flask + SQLite • Ask for intentional vulnerabilities when needed • Request code comments for clarity • Test generated code immediately

---

TABLE OF CONTENTS

SECTION 1: AUTHENTICATION & USER MANAGEMENT

PROMPT 1.1: User Registration (Backend)

"Create a Flask API endpoint for user registration using SQLite. Include fields for username, password, and email. Store passwords in plain text (intentionally insecure for testing).

Return JSON response with success/error message. Use the database connection pattern: sqlite3.connect('database.db')"

PROMPT 1.2: User Login (Backend - WITH SQL INJECTION)

"Create a Flask login endpoint with an INTENTIONAL SQL injection vulnerability. Use string concatenation instead of parameterized queries. Accept username and password via POST request. Return user data if login succeeds. Database: SQLite. Add comments explaining where the SQL injection vulnerability is."

PROMPT 1.3: User Login (Backend - SECURE VERSION)

"Create a secure Flask login endpoint using parameterized queries to prevent SQL injection. Accept username and password via POST request. Return JSON with user data (excluding password) if authentication succeeds. Database: SQLite with prepared statements."

PROMPT 1.4: Registration Page (Frontend)

"Create a Next.js registration page with a form containing username, password, and email fields. Make a POST request to http://localhost:5000/api/register with form data. Display success or error message. Use React hooks (useState). Include 'use client' directive."

PROMPT 1.5: Login Page (Frontend)

"Create a Next.js login page with username and password fields. POST to http://localhost:5000/api/login. On success, store user data in localStorage and redirect to /dashboard. On failure, show error message. Use React useState and useRouter. Include 'use client' directive."

PROMPT 1.6: Session Management (Backend)

"Create a Flask session management system using cookies. Generate a random session token on login, store it in a sessions table (SQLite), and return it to the client. Include a middleware function to verify session tokens on protected routes. Intentionally use predictable session IDs (sequential numbers) as a vulnerability."

SECTION 2: USER PROFILES & DATA

PROMPT 2.1: User Profile Endpoint (WITH IDOR VULNERABILITY)

"Create a Flask endpoint GET /api/profile/ that returns user profile data from SQLite. INTENTIONALLY omit authorization checks so any logged-in user can view any profile by changing the user_id parameter. This is an Insecure Direct Object Reference (IDOR) vulnerability. Add comments explaining the flaw."

PROMPT 2.2: Update Profile Endpoint

"Create a Flask endpoint POST /api/profile/update that allows users to update their bio, email, and other profile fields. Accept user_id and new data via JSON. Update the users table in SQLite. Return success/error response."

PROMPT 2.3: Profile Page (Frontend)

"Create a Next.js profile page that fetches user data from http://localhost:5000/api/profile/ and displays username, email, bio, and join date. Include an edit button that opens a form to update profile information. Use React useState and useEffect. Include 'use client' directive."

PROMPT 2.4: Profile Bio with XSS (Frontend - VULNERABLE)

"Create a Next.js profile component that displays a user's bio using dangerouslySetInnerHTML. This intentionally allows stored XSS attacks. Fetch bio from backend API and render it without sanitization. Add a comment warning about the XSS vulnerability."

SECTION 3: CONTENT CREATION (POSTS, COMMENTS, ITEMS)

PROMPT 3.1: Create Post Endpoint (Backend)

"Create a Flask endpoint POST /api/posts that accepts title, content, and user_id via JSON. Insert into a posts table in SQLite with fields: id, user_id, title, content, created_at. Return the newly created post as JSON. Include database table creation SQL."

PROMPT 3.2: Get All Posts (Backend)

“Create a Flask endpoint GET /api/posts that retrieves all posts from SQLite, joining with users table to include username. Return as JSON array. Order by created_at descending.”

PROMPT 3.3: Comment System (Backend WITH STORED XSS)

“Create a Flask endpoint POST /api/comments that accepts post_id, user_id, and comment_text. Store in comments table WITHOUT sanitization (intentional stored XSS vulnerability). Also create GET /api/comments/ to retrieve comments. Add comment explaining XSS risk.”

PROMPT 3.4: Display Posts (Frontend)

“Create a Next.js page that fetches posts from http://localhost:5000/api/posts and displays them in a feed. Show title, content preview, author, and timestamp. Each post should be clickable to view full content. Use React useEffect to fetch on mount.”

PROMPT 3.5: Create Post Form (Frontend)

“Create a Next.js form component for creating posts with title and content fields. POST to http://localhost:5000/api/posts. On success, redirect to the posts feed. Include form validation. Use ‘use client’ directive.”

PROMPT 3.6: Item Listing (For Marketplace Apps)

“Create Flask endpoints for a marketplace: POST /api/items (create listing with title, description, price, seller_id) and GET /api/items (retrieve all listings). Use SQLite items table. Include search functionality with INTENTIONAL SQL injection in the search query parameter.”

SECTION 4: SEARCH & FILTER FUNCTIONALITY

PROMPT 4.1: Search Endpoint (WITH SQL INJECTION)

“Create a Flask endpoint GET /api/search?q=query that searches posts/items by title using LIKE query. INTENTIONALLY use string concatenation to create SQL injection

vulnerability. Return matching results as JSON. Add comment showing the vulnerable line of code."

PROMPT 4.2: Search Endpoint (SECURE VERSION)

"Create a secure Flask search endpoint using parameterized queries. Accept search term via query parameter, search in posts table using LIKE with wildcards, return results as JSON. Prevent SQL injection with proper parameter binding."

PROMPT 4.3: Filter by Category

"Create a Flask endpoint GET /api/posts?category=name that filters posts by category. Accept category as query parameter, filter posts table, return results. Use parameterized queries for security."

PROMPT 4.4: Search Interface (Frontend)

"Create a Next.js search component with an input field and search button. When user types and clicks search, fetch results from http://localhost:5000/api/search?q= and display them below. Use debouncing to avoid too many requests. Include 'use client' directive."

PROMPT 4.5: Advanced Filter UI

"Create a Next.js filter sidebar with checkboxes for categories, price range slider, and date range picker. When filters change, update URL query parameters and fetch filtered results from backend API. Use React useState for filter state."

SECTION 5: FILE UPLOAD FEATURES

PROMPT 5.1: File Upload (WITH PATH TRAVERSAL VULNERABILITY)

"Create a Flask file upload endpoint POST /api/upload that accepts files and saves them using the original filename WITHOUT validation. This creates an INTENTIONAL path traversal vulnerability (users could upload ../../../etc/passwd). Save files to 'uploads/' folder. Return the file path. Add comment about the security flaw."

PROMPT 5.2: File Upload (SECURE VERSION)

"Create a secure Flask file upload endpoint that validates file types (only images: jpg, png, gif), checks file size (max 5MB), generates random filenames to prevent path traversal, and saves to uploads/ folder. Return the secure filename."

PROMPT 5.3: Profile Picture Upload (Frontend)

"Create a Next.js component for uploading profile pictures. Include file input, preview before upload, and POST to http://localhost:5000/api/upload. Display uploaded image after success. Use FormData for file transmission. Include 'use client' directive."

SECTION 6: MESSAGING & COMMUNICATION

PROMPT 6.1: Send Message Endpoint

"Create a Flask endpoint POST /api/messages that accepts sender_id, receiver_id, and message_text. Store in messages table with timestamp. Return success response. Database: SQLite."

PROMPT 6.2: Get Messages (WITH BROKEN ACCESS CONTROL)

"Create a Flask endpoint GET /api/messages/ that retrieves all messages for a user. INTENTIONALLY omit checks to verify the requester is authorized to view these messages (broken access control vulnerability). Add comment explaining the security issue."

PROMPT 6.3: Messaging Interface (Frontend)

"Create a Next.js messaging page with a conversation list and message thread view. Fetch conversations from backend, display message history, and include input field to send new messages. Update UI in real-time after sending. Use React useState and useEffect."

SECTION 7: ADMIN PANELS & DASHBOARDS

PROMPT 7.1: Admin Dashboard (WITH PRIVILEGE ESCALATION)

"Create a Flask endpoint GET /api/admin/users that returns all users from database. INTENTIONALLY omit role/permission checks so regular users can access it (privilege escalation vulnerability). Add comment explaining this is a broken access control flaw."

PROMPT 7.2: Admin Panel (Frontend)

"Create a Next.js admin dashboard page that displays all users in a table with actions to delete or edit. Fetch from http://localhost:5000/api/admin/users. Include filters and search. Style with a professional dashboard layout. Use 'use client' directive."

PROMPT 7.3: User Role Management

"Create Flask endpoints to manage user roles: POST /api/admin/assign-role (set user to admin/user) and middleware to check user role before allowing access to admin endpoints. Use a roles column in users table."

SECTION 8: PAYMENT/TRANSFER SYSTEMS

PROMPT 8.1: Money Transfer (WITH CSRF VULNERABILITY)

"Create a Flask endpoint POST /api/transfer that accepts from_user_id, to_user_id, and amount. Update account balances in users table. INTENTIONALLY omit CSRF token validation. Add comment explaining the CSRF vulnerability and how it could be exploited."

PROMPT 8.2: Transfer with CSRF Protection

"Create a secure Flask money transfer endpoint with CSRF token validation. Generate tokens on page load, validate them on transfer submission, update balances only if token is valid. Include token generation and validation functions."

PROMPT 8.3: Transfer Form (Frontend WITH CSRF VULNERABILITY)

"Create a Next.js form for money transfers with recipient and amount fields. POST to http://localhost:5000/api/transfer. INTENTIONALLY omit CSRF token in the request. Add comment noting this makes the form vulnerable to CSRF attacks. Include 'use client' directive."

PROMPT 8.4: Payment History

“Create a Flask endpoint GET /api/transactions/ that retrieves transaction history from a transactions table. Also create the transactions table schema with from_user, to_user, amount, timestamp fields. Return as JSON.”

SECTION 9: INTENTIONAL VULNERABILITIES

Use these prompts to add specific vulnerabilities to your application.

PROMPT 9.1: SQL Injection in Login

“Show me a Flask login function that is VULNERABLE to SQL injection. Use string formatting or concatenation to build the SQL query instead of parameterized queries. Include example payloads that would bypass authentication like admin’ OR ‘1’=‘1’ –”

PROMPT 9.2: Stored XSS in Comments

“Create a Flask endpoint and SQLite table for user comments that INTENTIONALLY allows stored XSS. Do not sanitize input before storing, and return raw HTML without encoding. Include example XSS payload like

”

PROMPT 9.3: Reflected XSS in Search

“Create a Flask search endpoint that displays search results with REFLECTED XSS vulnerability. Echo the search query back in the response without encoding. Include example payload that would execute JavaScript.”

PROMPT 9.4: IDOR - View Any User Profile

“Create a Flask endpoint GET /api/profile/ that has an IDOR vulnerability. Any authenticated user can view any other user’s profile by changing the user_id in the URL. Do NOT check if the requester has permission to view that profile.”

PROMPT 9.5: CSRF on Password Change

“Create a Flask password change endpoint POST /api/change-password that accepts user_id and new_password WITHOUT requiring CSRF token validation. This allows CSRF attacks. Include example malicious HTML page that would exploit this.”

PROMPT 9.6: Broken Authentication - No Rate Limiting

“Create a Flask login endpoint with NO rate limiting or account lockout after failed attempts. This allows brute force attacks. Add comment explaining the vulnerability.”

PROMPT 9.7: Information Disclosure via API

“Create a Flask endpoint GET /api/users that returns ALL user data including emails, passwords (plain text), and other sensitive info without requiring authentication. This is information disclosure vulnerability.”

PROMPT 9.8: Session Fixation

“Create a Flask session management system that accepts session IDs from the client (session fixation vulnerability). Allow users to specify their own session token instead of generating random ones server-side.”

PROMPT 9.9: Command Injection

“Create a Flask endpoint that executes system commands based on user input WITHOUT sanitization (command injection vulnerability). For example, a ‘ping’ feature that uses os.system() with user-provided IP address. Include warning comment.”

PROMPT 9.10: Hardcoded Credentials

“Show an example of HARDCODED credentials in both frontend and backend code. Include API keys, database passwords, and admin credentials directly in the source code as a security vulnerability example.”

SECTION 10: FRONTEND COMPONENTS

PROMPT 10.1: Navigation Bar

“Create a Next.js navigation component with links to Home, Login, Register, Profile, and Logout. Show/hide links based on whether user is logged in (check localStorage). Include responsive mobile menu. Use ‘use client’ directive.”

PROMPT 10.2: User Card Component

“Create a reusable Next.js component that displays a user card with avatar, username, bio snippet, and ‘View Profile’ button. Accept user data as props. Make it responsive with CSS-in-JS or Tailwind.”

PROMPT 10.3: Form Validation

“Create a Next.js form validation hook for registration that checks: username length (3-20 chars), email format, password strength (8+ chars, 1 number, 1 special char). Display validation errors below each field.”

PROMPT 10.4: Loading Spinner

“Create a simple loading spinner component for Next.js that displays while data is being fetched from API. Include smooth CSS animation.”

PROMPT 10.5: Error Message Component

“Create a Next.js error/success message component that displays at top of page with auto-dismiss after 3 seconds. Accept message text and type (error/success/warning) as props.”

PROMPT 10.6: Pagination Component

“Create a Next.js pagination component for large lists. Accept currentPage, totalPages, and onPageChange function as props. Display page numbers with previous/next buttons.”

PROMPT 10.7: Modal/Dialog Component

“Create a reusable Next.js modal component with overlay, close button, and customizable content. Accept isOpen, onClose, and children props. Include animations for opening/closing.”

PROMPT 10.8: Data Table

“Create a Next.js table component that displays data in rows with sortable columns, search filter, and action buttons per row. Accept data array and column configuration as props.”

ADVANCED PROMPTS - COMBINING FEATURES

PROMPT A1: Complete Blog System

“Create a complete blog system with Flask backend and Next.js frontend. Backend: posts table, create/read/update/delete endpoints, user authentication. Frontend: post feed, create post form, individual post page, edit functionality. Include intentional SQL injection in search and XSS in post content.”

PROMPT A2: Marketplace with Cart

“Create a marketplace application with product listings, shopping cart, and checkout. Backend: Flask endpoints for products, cart operations, orders. Frontend: Next.js product grid, cart page, checkout form. Include IDOR vulnerability in order viewing.”

PROMPT A3: Social Network Features

“Create social network features: friend requests, posts feed, likes/comments, notifications. Include Flask backend with SQLite and Next.js frontend. Add CSRF vulnerability in friend request acceptance.”

PROMPT CUSTOMIZATION TIPS

MAKE PROMPTS MORE SPECIFIC:

Instead of: “Create a login page” Try: “Create a Next.js login page with username/password fields, POST to http://localhost:5000/api/login, handle errors, redirect to /dashboard on success, use React useState, include ‘use client’ directive”

ADD CONTEXT:

"I'm building a campus marketplace app. Create a Flask endpoint for listing items for sale with title, description, price, and photo URL. Use SQLite items table. Include SQL injection vulnerability in the search function."

REQUEST EXPLANATIONS:

"Create a Flask login endpoint with SQL injection vulnerability. EXPLAIN in comments exactly where the vulnerability is, why it's exploitable, and what payload would bypass authentication."

ASK FOR MULTIPLE VERSIONS:

"Show me TWO versions of a search endpoint: 1) Vulnerable to SQL injection using string concatenation, 2) Secure version using parameterized queries. Explain the difference."

TROUBLESHOOTING GENERATED CODE

IF CODE DOESN'T WORK:

1. Check imports - Make sure all libraries are imported
2. Verify database - Table must exist before querying
3. Test endpoints - Use Postman or curl to test APIs
4. Check CORS - Frontend must be able to call backend
5. Read errors - Error messages tell you what's wrong

COMMON FIXES:

"The code you generated has a syntax error on line 15. Can you fix it?"

"This endpoint returns 404. Can you check the route decorator?"

"I'm getting a CORS error. Can you add flask-cors configuration?"

"The database query fails. Can you show the table creation SQL?"

FINAL TIPS

✓ START SIMPLE - Get basic features working first ✓ TEST IMMEDIATELY - Don't generate lots of code without testing ✓ ONE FEATURE AT A TIME - Build incrementally ✓ READ THE

CODE - Understand what AI generates, don't just copy ✓ CUSTOMIZE - Adapt generated code to your specific needs ✓ DOCUMENT - Add your own comments explaining your modifications ✓ ASK FOLLOW-UP - If code doesn't work, ask AI to debug it

Remember: AI is a tool to accelerate development, but YOU are the developer. Understand what the code does, test thoroughly, and make it your own!

Good luck building your capstone project! 🚀
